

Ultra-Fast Boolean Evaluation with Correspondence Matrices: Algorithms, Theory, and Empirical Evaluation

Brian Droncheff
Brian@B-Theory.com

Abstract

We present a compact, matrix-mechanics formulation of propositional logic—Correspondence Matrices (CMs)—together with practical algorithms that evaluate and combine Boolean expressions using only bit-level (binary) operations. The central identity (??) turns algebra over expressions into algebra over tiny 2×2 binary operators, enabling aggressive fusion and broadcast-only materialization. Benchmarks show large speedups over standard approaches; we summarize setup and headline results in ??. Formal proofs and extended tables appear in ????.

1 Introduction

Problem. Evaluating and combining large sets of Boolean expressions underpins hardware verification, logic synthesis, SAT pipelines, and compilers. Traditional engines (symbolic simplification, BDDs, Espresso) are powerful but often memory- or pointer-heavy.

Idea. We recast each binary Boolean operator as a binary 2×2 correspondence matrix (CM). Expressions become bra-ket measurements $\langle X || [\Theta] Y \rangle$, and composition of expressions reduces to pointwise composition of their operator matrices, so the hot path is bitwise ($\wedge, \vee, m, \neg$) and vectorizable.

Contributions. (i) A compact operator calculus centered on the operator-composition “hinge” identity (??) and linearity (??); (ii) a lazy CM compiler that aligns by broadcast and materializes once (??); (iii) an empirical study showing large speedups versus BDD and SymPy backends (??).

Where figures/proofs go. Key identities are stated inline and proved succinctly in ??. Figures illustrating CM transforms (rotation/transpose/negation) and the operator table appear in ????. Benchmark plots (time vs. depth; speedups) are in ????.

2 Background and Notation

Let $\langle X | = [X, \neg X]$ and $|Y\rangle = [Y, \neg Y]^\top$. A CM is a 2×2 binary matrix $[\Theta]$ such that

$$X\Theta Y = \langle X || [\Theta] Y \rangle.$$

The 16 binary operators correspond to the 16 binary 2×2 CMs; basis elements are the rank-1 indicators $|1\rangle\langle 1|$, $|1\rangle\langle 0|$, $|0\rangle\langle 1|$, $|0\rangle\langle 0|$ (see ??). Transformations of operators (negation, swap, literal complements) act as simple matrix transforms (??).

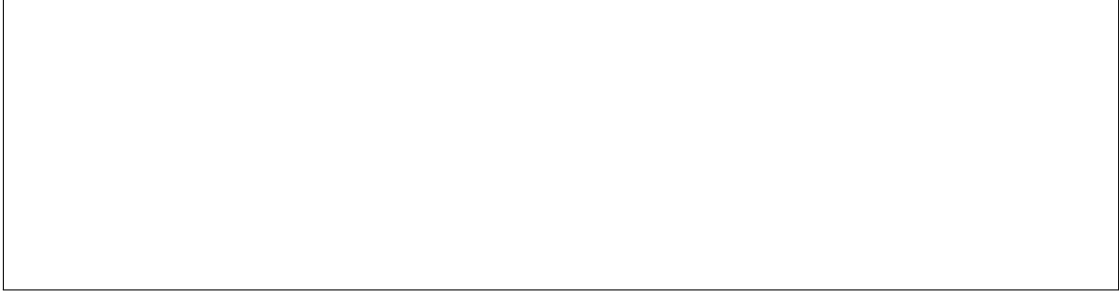


Figure 1: CM transform effects (transpose/rotation/negation) on the 2×2 operator grid; used to normalize operands before applying (1).

3 The Correspondence–Matrix Calculus

3.1 Operator–composition (the hinge identity)

$$\langle X \parallel [\Theta_1] \parallel Y \rangle \Phi \langle X \parallel [\Theta_2] \parallel Y \rangle = \langle X \parallel [\Theta_1] \Phi [\Theta_2] \parallel Y \rangle. \quad (1)$$

Equivalently, $(X\Theta_1 Y) \Phi (X\Theta_2 Y) = X (\Theta_1 \Phi \Theta_2) Y$. A short proof is given in ??.

3.2 Transform rules via matrix operations

Negation, swapping operands, and literal complements become CM transforms, e.g.

$$\neg \langle X \parallel [\Theta] Y \rangle = \langle X \parallel \neg[\Theta] Y \rangle, \quad \langle Y \parallel [\Theta] X \rangle = \langle X \parallel [\Theta]^\top Y \rangle,$$

and similar rotation/transpose identities used to normalize sub-expressions so that (1) applies (??).

3.3 Linearity and basis decomposition

CMs are linear over XOR:

$$\langle X \parallel ([\Theta_1] \mathfrak{m} [\Theta_2]) Y \rangle = \langle X \parallel [\Theta_1] Y \rangle \mathfrak{m} \langle X \parallel [\Theta_2] Y \rangle.$$

Thus any operator decomposes in the four-element basis: $[\Theta] = \Theta_{11}[\wedge] \mathfrak{m} \Theta_{12}[\uparrow] \mathfrak{m} \Theta_{21}[\downarrow] \mathfrak{m} \Theta_{22}[\neg\vee]$. A compact table appears in ??.

4 From CMs to Logical Measurement Matrices

We lift to logical measurement matrices (LMs), e.g. $[M_{X\Theta Y}] = \Theta_{ij} |X_i\rangle\langle Y_j|$, whose positive valuation recovers $[\Theta]$. Measurements such as

$$\langle L_1 \parallel [M_{X\Theta Y}] \parallel L_2 \rangle = \Theta_{rs} (L_1 \Leftrightarrow X_r) \wedge (Y_s \Leftrightarrow L_2) \quad (2)$$

make constraints explicit and motivate the analogy with quantum measurement.

5 Algorithms and Implementation

Two evaluation strategies. (A) *Operator fusion at 2×2 scale*: normalize sub-expressions then repeatedly apply (1), composing only 2×2 CMs until the final measurement; (B) *Block path*: build 4×4 (or $2^n \times 2^n$) blocks with small modifiers, compose by indices (still bitwise), and measure once at the end. Pseudocode and micro-optimizations (broadcast alignment, LRU permutation caches) go here, with proofs deferred to ??.

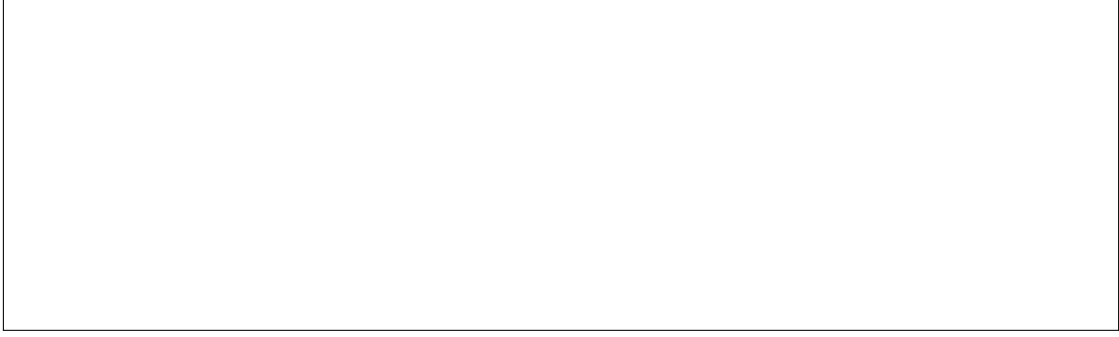


Figure 2: Runtime vs. expression depth at $n = 16$ (medians over 10 trials) for CM, BDD, SymPy.

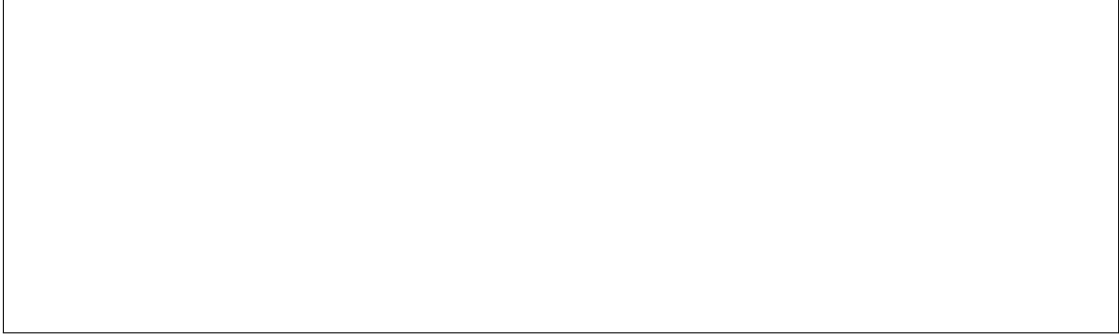


Figure 3: Speedups at $n = 16$: BDD/CM and SymPy/CM across depths 2–7.

Lazy CM compiler. Our implementation delays materialization: align by name using size-1 axis insertion and NumPy broadcasting; cache bit-permutation index arrays; materialize the canonical CM once at the end. This minimizes memory traffic on deep trees and keeps inner loops bitwise.

6 Empirical Evaluation

6.1 Setup

We generate random Boolean ASTs and compare backends: CM (lazy), SymPy, a tiny ROBDD, optional `dd.autoref` BDD, Espresso, and a canonical $TT \rightarrow DNF$ baseline. Correctness is checked by vectorized truth-table evaluation outside timing windows. (Benchmark harness summarized in the README.)

6.2 Headline results (medians)

Insert the plot/table derived from your HTML report:

- At $n = 16$, CM median speedups vs. `dd.autoref` BDD $\approx 79\times\text{--}334\times$ over depths 2–7.
- At $n = 16$, CM is $1.7\times\text{--}7.4\times$ faster than SymPy for depths ≥ 3 ; SymPy slightly ahead at depth 2.

Correctness summary. CM passed all runs in the sweep shown; ROBDD and BDD?SOP baselines match the ground truth by construction where enabled; Espresso correctness flags pending integration fixes for some settings.

7 Discussion

Why it is fast: (1) fuses work at the 2×2 scale; lazy materialization avoids intermediate tables; broadcasting/cached permutations prevent quadratic blowups on deep trees. Other benefits: (i) purely binary arithmetic; (ii) closed, linear operator algebra; (iii) measurement-operator analogy; (iv) SIMD/GPU friendliness.

8 Conclusion

Correspondence Matrices turn Boolean evaluation into small, linear operator algebra plus bitwise kernels, yielding large empirical gains at higher variable counts.

Reproducibility. Include a small code block or link to your CLI invocation for the sweep (e.g., the depth sweep across $n \in \{4, 8, 16\}$ and 10 trials).

A Short Proofs and Derivations

A.1 Operator-composition (Eq. 1)

Using ANF, $\langle X \parallel [\Theta] Y \rangle = \sum_{i,j} X_i \Theta_{ij} Y_j$ with XOR as addition. Then

$$(X\Theta_1 Y) \Phi (X\Theta_2 Y) = \sum_{i,j} X_i (\Theta_1)_{ij} Y_j \Phi \sum_{k,\ell} X_k (\Theta_2)_{k\ell} Y_\ell,$$

distribute over XOR, use $X_i X_k = \delta_{ik} X_i$ and $Y_j Y_\ell = \delta_{j\ell} Y_j$, and note $(\Theta_1)_{ij} \Phi (\Theta_2)_{ij} = (\Theta_1 \Phi \Theta_2)_{ij}$ to obtain (1).

A.2 Linearity and basis decomposition

Elementwise XOR linearity follows immediately from the binary field structure. Basis expansion uses the four rank-1 indicators $|1\rangle\langle 1|$, $|1\rangle\langle 0|$, $|0\rangle\langle 1|$, $|0\rangle\langle 0|$.

B Operator Tables and Transform Rules

Placeholder for a compact 2×2 operator table and a schematic of transpose/rotation/negation effects. (This is where you can paste a single figure that shows all 16 CMs and their complements, plus the rotation/transpose relationships.)

C Additional Experimental Plots

Suggestions: (i) per-depth violin plots of per-trial times, (ii) correctness tallies, (iii) dd.autoref/ROBDD node counts vs. depth, (iv) sensitivity to variable ordering (if applicable).

D Implementation Notes (Extended)

Details on AST, broadcasting scheme, permutation caches, and backend toggles (concise bullets, with references to code).